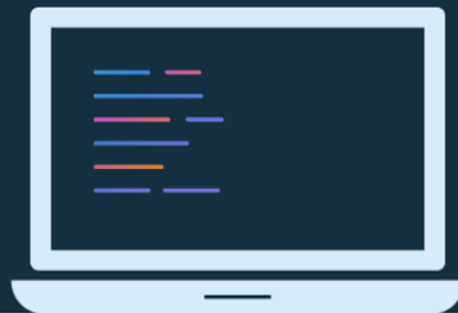




Android Development with Kotlin

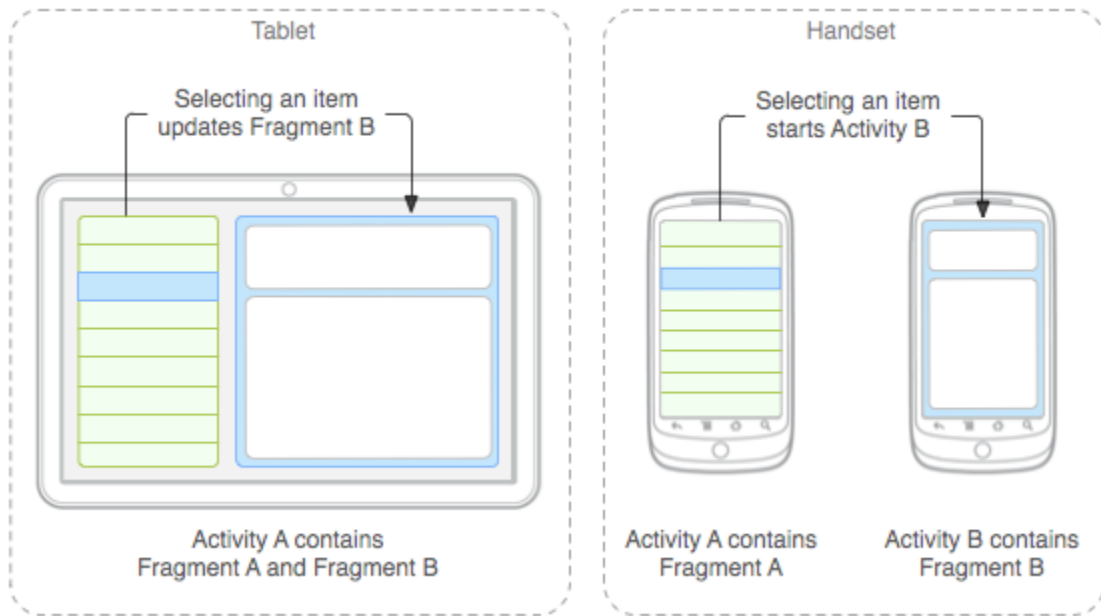


Fragment

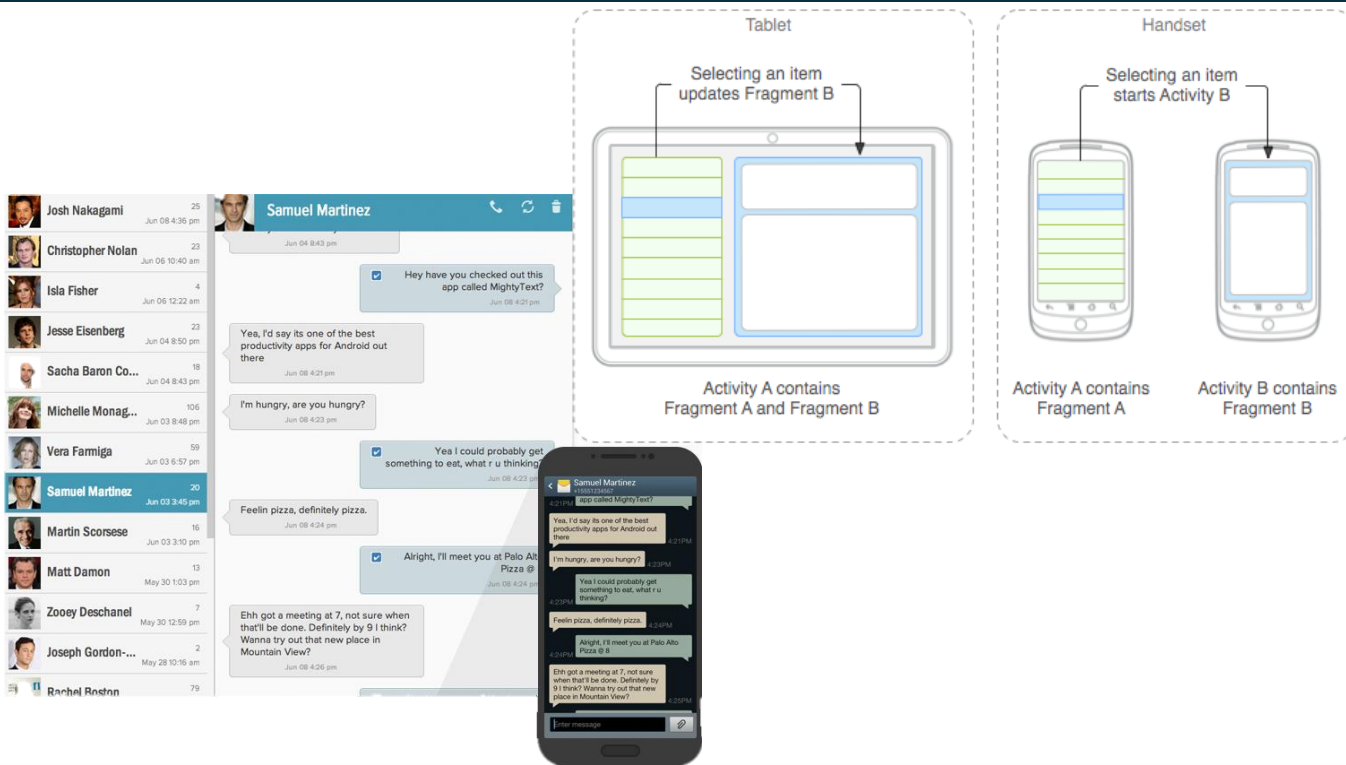
Fragment

- It represents a **behavior** or a **portion** of the user interface in an Activity.
- It has its own lifecycle, receives its own input events, and which you can add or remove while the activity is running.
- You can combine multiple fragments in a single activity to build a multi-pane UI and reuse a fragment in multiple activities.

Fragments for tablet layouts



Fragment Example



Fragment cont'd

- A fragment **must** always be embedded in an activity and the fragment's lifecycle is directly affected by the host activity's lifecycle.
 - when the activity is **paused**, so are all fragments in it.
 - when the activity is **destroyed**, so are all fragments.
 - However, while an activity is **running** (it is in the resumed lifecycle state), you can manipulate each fragment independently, such as **adding** or **removing** them.

Fragment cont'd

- When you perform such a fragment transaction, you can also add it to a back stack that's managed by the activity
 - each back stack entry in the activity is a record of the fragment transaction that occurred.
- The back stack allows the user to reverse a fragment transaction (navigate backward), by pressing the Back button.

Fragment cont'd

- Represents a behavior or portion of the UI in an activity ("microactivity")
- Must be hosted in an activity
- Lifecycle tied to host activity's lifecycle
- Can be added or removed at runtime

Fragment cont'd

- Fragment can be added to the Activity's layout via `<FragmentManager>` or from your application code by adding it to an existing `ViewGroup`.
- However, a fragment is not required to be a part of the activity layout; you may also use a fragment without its own UI as an invisible worker for the activity.

Fragment cont'd

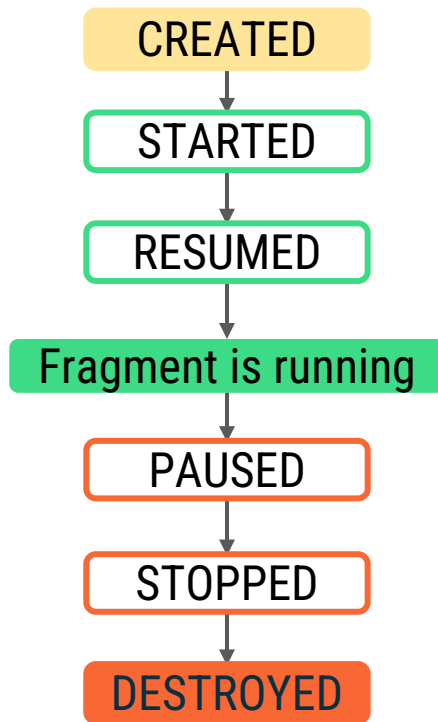
- Android introduced fragments in Android 3.0 (API level 11)
- Prior to the fragment introduction, we had a limitation because we can show only a single activity on the screen at one given point in time.
- So, we were not able to divide device screen and control different parts separately.

Fragment cont'd

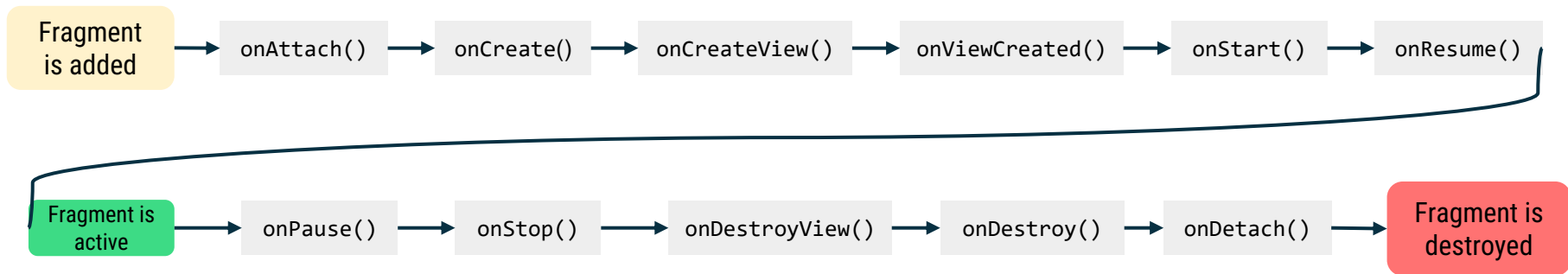
- But with the introduction of fragments, we got more flexibility and removed the limitation of having a single activity on the screen at a time.
- Now we can have a single activity, but each activity can comprise of multiple fragments which will have their own layout, events and complete lifecycle.

Fragment lifecycle

Fragment states



Fragment lifecycle diagram



onAttach()

- Called when a fragment is attached to a context
- Immediately precedes `onCreate()`

onCreate()

- The system calls this when creating the fragment.
- Within your implementation, you should initialize essential components of the fragment that you want to retain when the fragment is *paused* or *stopped*, then *resumed*.

onCreateView()

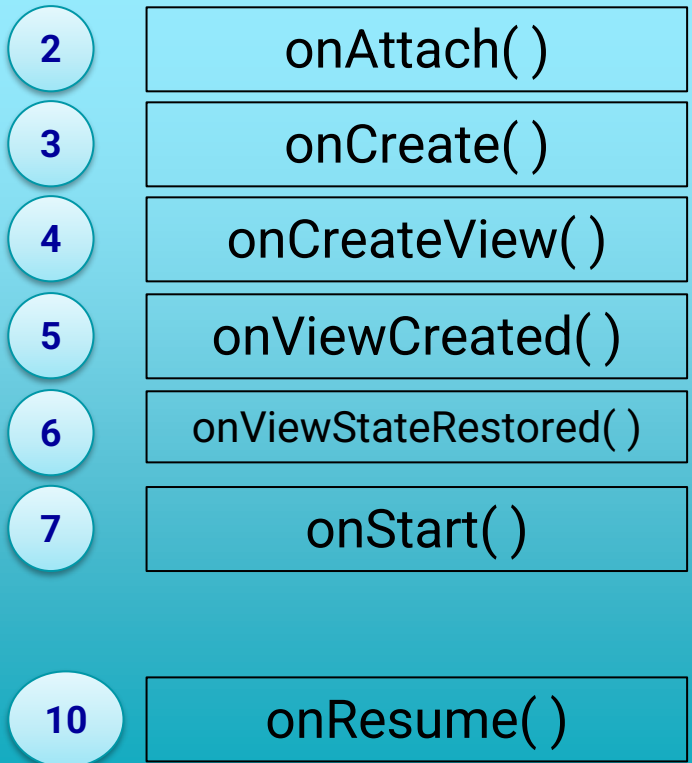
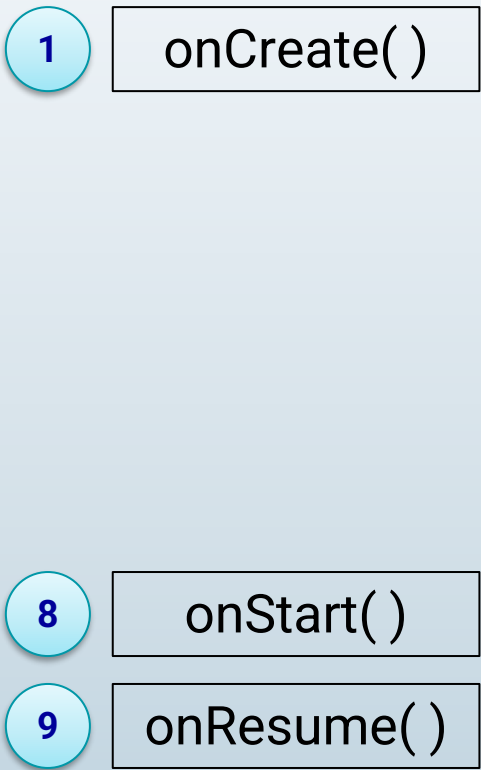
- Called to create the view hierarchy associated with the fragment
- The system calls this when it's time for the fragment to draw its user interface for the first time.
- To draw a UI for your fragment, you must return a View from this method that is the root of your fragment's layout.
You can return null if the fragment does not provide a UI.
- **Inflate** the fragment layout here and return the root view

onViewCreated()

- Called when view hierarchy has already been created
- Perform any remaining initialization here
(for example, restore state from `Bundle`)

Activity VS Fragment Lifecycle

Creation



onPause()

- The system calls this method as the first indication that the user is leaving the fragment (though it does not always mean the fragment is being destroyed).
- This is usually where you should commit any changes that should be persisted beyond the current user session (because the user might not come back)

onDestroyView() and onDetach()

- `onDestroyView()`
is called when view hierarchy of fragment is removed.
- `onDetach()`
is called when fragment is no longer attached to the host.

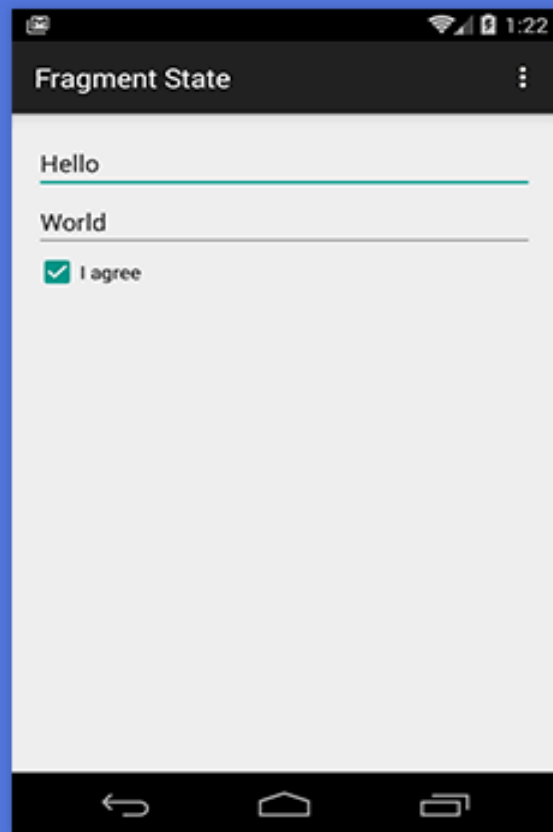
Fragment View State Saving/Restoring



Instance States



View States



Summary of fragment states

State	Callbacks	Description
Initialized	<code>onAttach()</code>	Fragment is attached to host.
Created	<code>onCreate()</code> , <code>onCreateView()</code> , <code>onViewCreated()</code>	Fragment is created and layout is being initialized.
Started	<code>onStart()</code>	Fragment is started and visible.
Resumed	<code>onResume()</code>	Fragment has input focus.
Paused	<code>onPause()</code>	Fragment no longer has input focus.
Stopped	<code>onStop()</code>	Fragment is not visible.
Destroyed	<code>onDestroyView()</code> , <code>onDestroy()</code> , <code>onDetach()</code>	Fragment is removed from host.

Save fragment state across config changes

Preserve UI state in fragments by storing state in `Bundle`:

- `onSaveInstanceState(outState: Bundle)`

Retrieve that data by receiving the `Bundle` in these fragment callbacks:

- `onCreate()`
- `onCreateView()`
- `onViewCreated()`

Lifecycle-aware components

Lifecycle-aware components

Adjust their behavior based on activity or fragment lifecycle

- Use the `androidx.lifecycle` library
- `Lifecycle` tracks the lifecycle state of an activity or fragment
 - Holds current lifecycle state
 - Dispatches lifecycle events (when there are state changes)

LifecycleOwner

- Interface that says this class has a lifecycle
- Implementers must implement `getLifecycle()` method

Examples: `Fragment` and `AppCompatActivity`
are implementations of `LifecycleOwner`

LifecycleObserver

Implement `LifecycleObserver` interface:

```
class MyObserver : LifecycleObserver {  
    @OnLifecycleEvent(Lifecycle.Event.ON_RESUME)  
    fun connectListener() {  
        ...  
    }  
}
```

Add the observer to the lifecycle:

```
myLifecycleOwner.getLifecycle().addObserver(MyObserver())
```

Tasks and back stack

Understanding the back stack

State 1



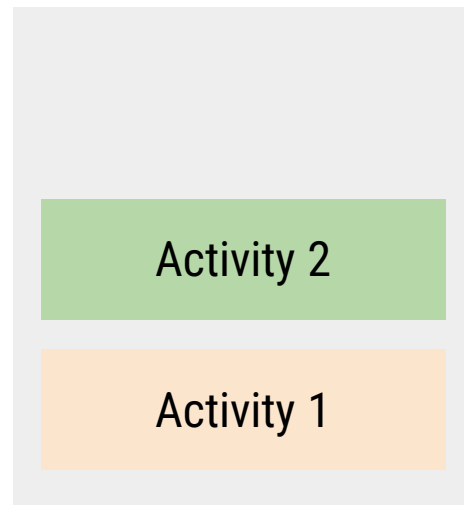
Back stack

State 2



Back stack

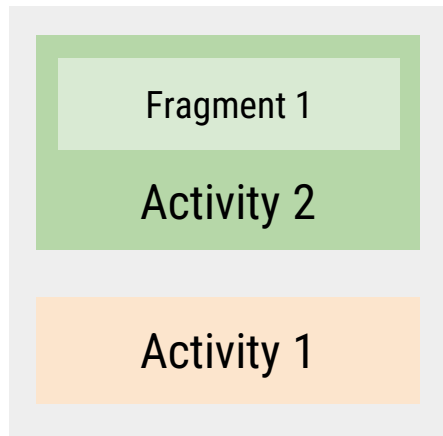
State 3



Back stack

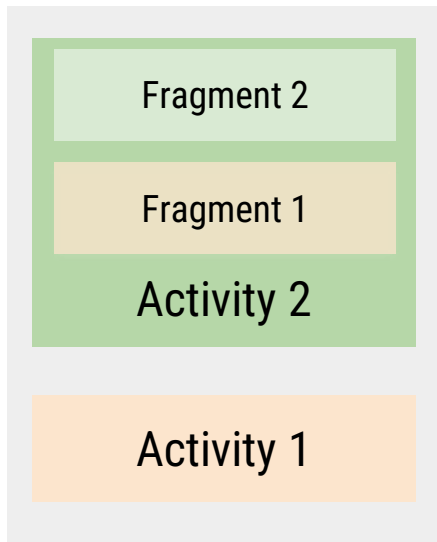
Fragments and the back stack

State 1



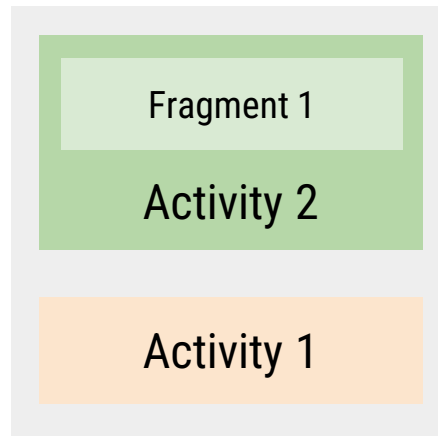
Back stack

State 2



Back stack

State 3



Back stack

First destination in the back stack



FirstFragment

Back stack



Add a destination to the back stack

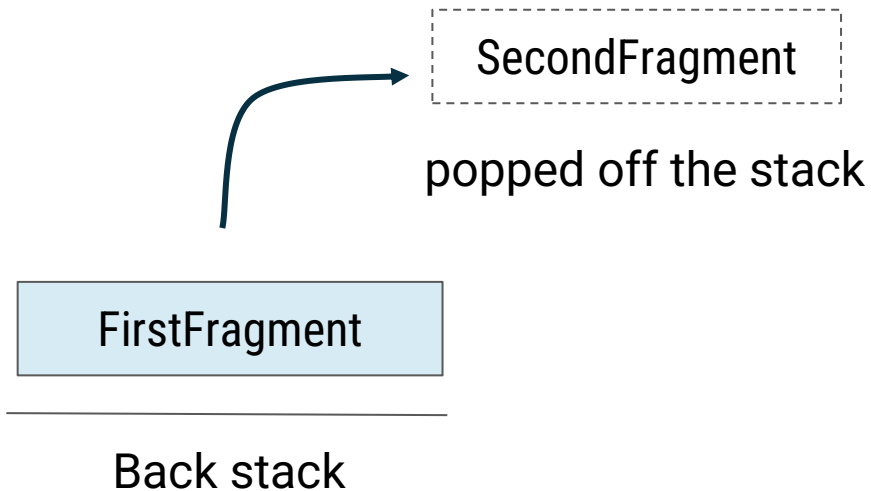


SecondFragment

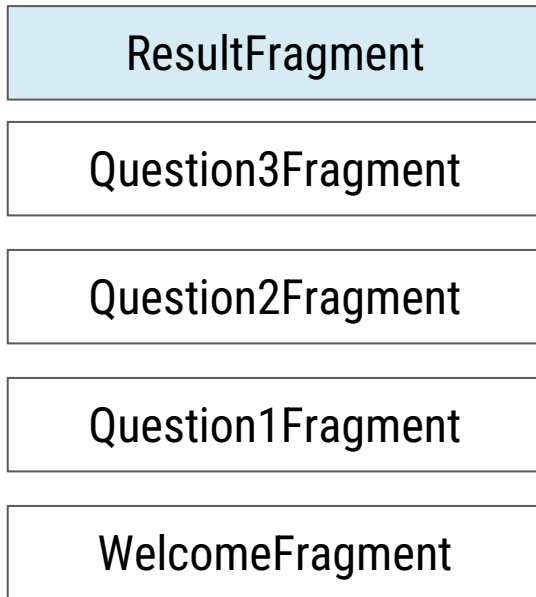
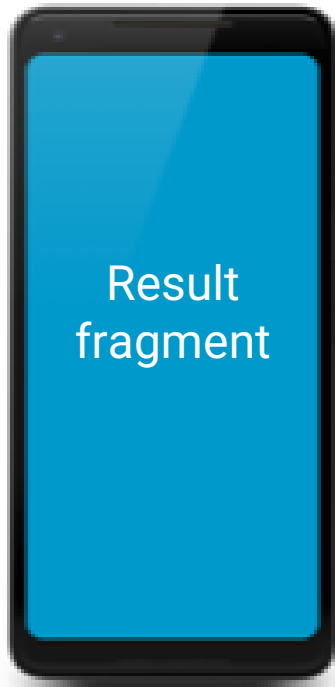
FirstFragment

Back stack

Tap Back button



Another back stack example

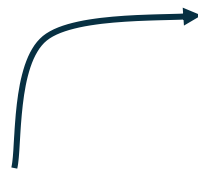


Back stack

Modify Back button behavior



pop additional destinations
off the back stack



ResultFragment

Question3Fragment

Question2Fragment

Question1Fragment

WelcomeFragment

Back stack



Create Fragment

How to Create Fragment?

- Create a subclass of Fragment (or an existing subclass of it)
- Override the suitable callback methods.
- Usually, you should implement at least the following lifecycle methods:
 - onCreate()
 - onCreateView()
 - onViewCreated()
 - onPause()

Note about fragments

Use the AndroidX version of the `Fragment` class.
(`androidx.fragment.app.Fragment`).

Don't use the platform version of the `Fragment` class
(`android.app.Fragment`), which was deprecated.

Adding Fragments to Activity

Two Ways to Add Fragments to an Activity:

1. Add `<Fragment>` to the XML Layout
2. Add Fragment in Code

1- Adding Fragment to XML

Adding Fragment to XML Steps

1. Create a separate layout file that contains the appearance.
2. Create a class that extends `Fragment`
3. Use the `onCreateView()` method to link its appearance.
4. Attach the fragment inside the Activity through XML using `<fragment>` or `<FragmentContainerView>`

Any number of fragments can be attached to an Activity

1- Create Layout File

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".StaticFragment">

    <!-- TODO: Update blank fragment layout -->
    <TextView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:text="Hello blank fragment" />

</FrameLayout>
```

fragment_static.xml



2- Extend the Fragment

```
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment

class StaticFragment : Fragment() {
```

StaticFragment.kt

3- Link the Appearance

```
override fun onCreateView(  
    inflater: LayoutInflater,  
    container: ViewGroup?,  
    savedInstanceState: Bundle?  
): View? {  
    // Inflate the layout for this fragment  
    return inflater.inflate(R.layout.fragment_static, container, attachToRoot: false)  
}
```

StaticFragment.kt

4- Attach the Fragment inside the Activity

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/fragmentContainerView"
        android:name="com.example.myapplication.StaticFragment"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

activity_main.xml

2- Adding Fragment to code

Fragment Manager

- Each Activity has its own Fragment Manager.
- Fragment Manager is accessible through:
 - `getFragmentManager()`
 - `getSupportFragmentManager()`
- It maintains references to all fragments inside the activity.
- To get a reference to a particular fragment:
 - `findFragmentById()`
 - `findFragmentByTag()`

Fragment Transactions

- Fragments manipulations are done through `FragmentManager` like:
 - Adding
 - Removing
 - Replacing
- Don't forget to **commit** after finishing the transactions.

Adding Fragment to Code Steps

1. Create a separate layout file or an object that contains the appearance.
2. Create a class that extends `Fragment`
3. Use the `onCreateView()` method to link its appearance.
4. Activity Manipulation
 - a) Create an object of the `Fragment` subclass
 - b) Get a reference to `FragmentManager`
 - c) Begin Transaction (add the fragment dynamically to the container layout)
 - d) Commit

1- Create Layout File

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".DynamicFragment">

    <!-- TODO: Update blank fragment layout -->
    <TextView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:text="Hello blank fragment" />

</FrameLayout>
```

fragment_dynamic.xml

2- Extend the Fragment

```
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment

class DynamicFragment : Fragment() {
```

DynamicFragment.kt

3- Link the Appearance

```
override fun onCreateView(  
    inflater: LayoutInflater,  
    container: ViewGroup?,  
    savedInstanceState: Bundle?  
): View? {  
    // Inflate the layout for this fragment  
    return inflater.inflate(R.layout.fragment_dynamic, container, attachToRoot: false)  
}
```

DynamicFragment.kt

4- Activity Manipulation

MainActivity.kt

```
private lateinit var fragmentManager: FragmentManager
private lateinit var dynamicFragment: DynamicFragment
private lateinit var transaction: FragmentTransaction
```

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
```

```
fragmentManager = supportFragmentManager
```

b

```
if (savedInstanceState == null) {
```

```
dynamicFragment = DynamicFragment()
```

a

```
transaction = fragmentManager.beginTransaction()
```

c

```
transaction.setReorderingAllowed(true)
```

```
transaction.add(R.id.fragmentContainerView, dynamicFragment, tag: "dynamic_fragment")
```

```
transaction.commit()
```

d

```
} else {
```

```
dynamicFragment = fragmentManager.findFragmentByTag(tag: "dynamic_fragment") as DynamicFragment
```

```
}
```

```
}
```

4- Activity Manipulation cont'd

```
<?xml version="1.0" encoding="utf-8"?>  
<androidx.constraintlayout.widget.ConstraintLayout  
    xmlns:android="http://schemas.android.com/apk/res/android"  
    xmlns:app="http://schemas.android.com/apk/res-auto"  
    xmlns:tools="http://schemas.android.com/tools"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    tools:context=".MainActivity">
```

```
    <androidx.fragment.app.FragmentContainerView  
        android:id="@+id/fragmentContainerView"  
        android:layout_width="0dp"  
        android:layout_height="0dp"  
        app:layout_constraintBottom_toBottomOf="parent"  
        app:layout_constraintEnd_toEndOf="parent"  
        app:layout_constraintStart_toStartOf="parent"  
        app:layout_constraintTop_toTopOf="parent" />  
</androidx.constraintlayout.widget.ConstraintLayout>
```

activity_main.xml

Demo

Assignment 1

Create an Activity with a **static** fragment
and override all of the Fragment's lifecycle methods

Assignment 2

Create an Activity with a **dynamic** fragment
and override all of the Fragment's lifecycle methods

Assignment 3

Create an Activity with two fragments, **static & dynamic**.